

Type Safety & Documentation — Senior Developer Level

JavaScript dynamically typed:

```
let value = 10;  
  
value = "hello";  
  
value = true;
```

JavaScript এটা allow করে।

এটা flexibility দেয়, কিন্তু বড় application-এ problem তৈরি করতে পারে:

```
Wrong Type  
  ↓  
Wrong Assumption  
  ↓  
Wrong Logic  
  ↓  
Runtime Bug
```

Type safety-এর লক্ষ্য হলো:

```
Unknown / Broad Data  
  ↓  
Understand its shape  
  ↓  
Narrow the type  
  ↓  
Validate when necessary  
  ↓  
Use safely
```

এই section-এর চারটি core concept:

```
JSDoc  
  ↓  
Developer / IDE understanding
```

Type Narrowing

↓

Runtime checks → safer code reasoning

Discriminated Unions

↓

Predictable object variants

Runtime Validation

↓

Untrusted external data → trusted application data

89. JSDoc Type Annotations

JSDoc কী?

JSDoc হলো JavaScript code-এর সাথে structured documentation লেখার একটি standard-style syntax।

সবচেয়ে interesting বিষয় হলো:

JavaScript project-এ TypeScript ব্যবহার না করেও JSDoc দিয়ে editor/type checker-কে অনেক type information দেওয়া যায়।

উদাহরণ:

```
/**
 * @param {string} name
 * @returns {string}
 */
function greet(name) {
  return `Hello ${name}`;
}
```

এখানে আমরা বলছি:

```
name → string
return → string
```

VS Code-এর মতো editor এই information ব্যবহার করে autocomplete এবং type checking improve করতে পারে।

Basic JSDoc

@param

```
/**
 * @param {string} name
 * @param {number} age
 */
function createUser(name, age) {
  return {
    name,
    age
  };
}
```

@returns

```
/**
 * @param {number} a
 * @param {number} b
 * @returns {number}
 */
function add(a, b) {
  return a + b;
}
```

Object type

ধরো:

```
/**
 * @param {{name: string, age: number}} user
 */
function printUser(user) {
  console.log(user.name);
  console.log(user.age);
}
```

এটা কাজ করে, কিন্তু বড় object-এর জন্য inline type ugly হয়ে যায়।

তখন `@typedef` ব্যবহার করা ভালো।

@typedef

```
/**
 * @typedef {Object} User
 * @property {string} name
 * @property {number} age
 * @property {boolean} active
 */
```

তারপর:

```
/**
 * @param {User} user
 */
function printUser(user) {
  console.log(user.name);
}
```

Mental model:

```
@typedef
  ↓
নিজের একটি reusable type description
  ↓
@param / @returns / variable
```

Optional property

```
/**
 * @typedef {Object} User
 * @property {string} name
 * @property {number} [age]
 */
```

এখানে:

```
name → required  
age  → optional
```

Array types

```
/**  
 * @param {string[]} names  
 */  
function printNames(names) {  
  names.forEach(name => {  
    console.log(name);  
  });  
}
```

অথবা:

```
/**  
 * @param {Array<string>} names  
 */
```

দুটোই দেখা যায়।

Union types

একটি value একাধিক type হতে পারে:

```
/**  
 * @param {string | number} value  
 */  
function printValue(value) {  
  console.log(value);  
}
```

এখন:

```
printValue("hello");  
printValue(100);
```

valid হতে পারে।

Literal types

JSDoc দিয়ে নির্দিষ্ট allowed value-ও document করা যায়:

```
/**  
 * @param {"admin" | "user" | "guest"} role  
 */  
function setRole(role) {  
  // ...  
}
```

এখানে arbitrary string নয়।

Allowed:

```
admin  
user  
guest
```

JSDoc + VS Code

VS Code JavaScript file-এ JSDoc ব্যবহার করলে editor:

```
Autocomplete  
Type hints  
Parameter hints  
Basic type checking
```

দিতে পারে।

আর আরও strict checking চাইলে `jsconfig.json` -এ:

```
{
  "compilerOptions": {
    "checkJs": true
  }
}
```

ব্যবহার করা যায়।

তারপর JavaScript file-এই type error detect করার capability বাড়ে।

JSDoc কখন useful?

বিশেষ করে:

- Legacy JavaScript project
- Node.js project
- Gradually migrating JS → TypeScript
- Shared utility functions
- Public/internal APIs
- Complex object structures
- Team documentation

JSDoc-এর limitation

JSDoc মূলত **static information/documentation** দেয়।

এটা runtime-এ:

```
const data = await fetch(url);
```

থেকে আসা data automatically validate করে না।

এই distinction খুব important:

JSDoc

↓

"আমি expect করছি data এমন হবে"

Runtime validation

↓
"বাস্তবে data এমন কি না check করছি"

90. Type Narrowing

JavaScript-এ আমরা অনেক সময় broad বা unknown data পাই।

যেমন:

```
function process(value) {  
  // value কী?  
}
```

এখন runtime check করে আমরা value-এর possible type narrow করতে পারি।

```
unknown / broad type  
  ↓  
runtime check  
  ↓  
narrowed type
```

typeof

সবচেয়ে basic type narrowing mechanism:

```
function printValue(value) {  
  if (typeof value === "string") {  
    console.log(value.toUpperCase());  
  }  
  
  if (typeof value === "number") {  
    console.log(value.toFixed(2));  
  }  
}
```

প্রথম block-এর মধ্যে আমরা logically জানি:


```
value → string
```

দ্বিতীয় block:

```
value → number
```

typeof results

Common values:

```
typeof "hello";    // "string"
typeof 10;          // "number"
typeof true;        // "boolean"
typeof undefined;   // "undefined"
typeof 10n;          // "bigint"
typeof Symbol();     // "symbol"
typeof function() {}; // "function"
typeof {};           // "object"
```

Important typeof null problem

```
typeof null
```

returns:

```
"object"
```

এটা JavaScript-এর historical behavior ।

তাই:

```
if (typeof value === "object") {
  // null-ও এখানে আসতে পারে
}
```

unsafe হতে পারে।

Better:

```
if (value !== null && typeof value === "object") {  
  // object  
}
```

instanceof

`instanceof` দিয়ে object কোনো constructor/class-এর instance কিনা check করা যায়।

```
class User {}  
  
const user = new User();  
  
console.log(user instanceof User);  
// true
```

Error handling example

```
try {  
  // ...  
} catch (error) {  
  if (error instanceof Error) {  
    console.log(error.message);  
  }  
}
```

এটা practical কারণ JavaScript-এ technically যেকোনো value throw করা যায়:

```
throw "Something went wrong";  
throw 123;  
throw { message: "Oops" };
```

তাই:

```
if (error instanceof Error)
```

একটি useful narrowing check।

Built-in objects

```
value instanceof Date  
value instanceof Map  
value instanceof Set  
value instanceof Error  
value instanceof URL
```

উদাহরণ:

```
function formatDate(value) {  
  if (value instanceof Date) {  
    return value.toISOString();  
  }  
  
  return "Not a Date";  
}
```

in operator

`in` দিয়ে object-এর মধ্যে property আছে কিনা check করা যায়।

```
const user = {  
  name: "Rahim",  
  age: 30  
};  
  
if ("name" in user) {  
  console.log(user.name);  
}
```

Discriminated Union-এর সাথে `in`

ধরো দুই ধরনের object:

```
const response = {  
  type: "success",  
  data: []  
};
```

অথবা:

```
const response = {  
  type: "error",  
  message: "Something went wrong"  
};
```

`in` দিয়ে:

```
function handleResponse(response) {  
  if ("data" in response) {  
    console.log(response.data);  
  }  
  
  if ("message" in response) {  
    console.log(response.message);  
  }  
}
```

`Array.isArray()`

Object check-এর ক্ষেত্রে এটা খুব important:

```
if (Array.isArray(value)) {  
  value.forEach(item => {  
    console.log(item);  
  });  
}
```

কারণ:

```
typeof []
```

returns:

```
"object"
```

তাই array detect করতে:

```
Array.isArray(value)
```

ব্যবহার করো।

Null + object check

একটি common pattern:

```
function process(value) {  
  if (  
    value !== null &&  
    typeof value === "object"  
  ) {  
    // object-like value  
  }  
}
```

তবে production application-এ শুধু এই check যথেষ্ট নয় যদি তোমার expected structure complex হয়।

User-defined type guard concept

যদি তুমি repeatedly একই object shape check করো, helper function বানাতে পারো।

```
function isUser(value) {  
  return (  
    value !== null &&  
    typeof value === "object" &&  
    typeof value.name === "string" &&  
    typeof value.age === "number"  
  )  
}
```

```
);  
}
```

তারপর:

```
if (isUser(data)) {  
  console.log(data.name);  
}
```

এটা runtime validation-এর basic form ।

Narrowing-এর Senior Mental Model

Type narrowing মানে:

আমি **runtime evidence**-এর মাধ্যমে সম্ভাব্য **type**-এর **range** ছোট করছি ।

```
unknown  
↓  
typeof === "string"  
↓  
string
```

অর:

```
unknown  
↓  
object  
↓  
"name" in object  
↓  
expected shape
```

91. Discriminated Unions Pattern in JavaScript

এটা বড় application-এর জন্য অত্যন্ত useful pattern ।

প্রথমে problem দেখি ।

ধরো API response তিন ধরনের হতে পারে:

```
success
error
loading
```

Naive object:

```
{
  status: "success",
  data: [...]
}
```

অন্যদিকে:

```
{
  status: "error",
  message: "Failed"
}
```

আর:

```
{
  status: "loading"
}
```

এখানে `status` হচ্ছে **discriminator** ।

Discriminator কী?

একটি property যেটা object-এর variant identify করে।

Common names:

```
type
kind
status
state
```

Example:

```
{
  type: "success",
  data: [...]
}
```

এখানে:

```
type = "success"
```

বলছে object কোন variant ।

Basic pattern

```
function handleResponse(response) {
  if (response.type === "success") {
    console.log(response.data);
    return;
  }

  if (response.type === "error") {
    console.error(response.message);
    return;
  }

  if (response.type === "loading") {
    console.log("Loading...");
  }
}
```

Mental model:

```
Response
├── type = success → data
├── type = error   → message
└── type = loading → no data
```

কেন এটা ভালো?

কারণ object-এর shape predictable হয়।

Bad design:

```
{
  status: "error",
  data: null,
  message: "Failed",
  loading: false,
  error: true
}
```

এখানে অনেক contradictory state possible।

যেমন:

```
{
  status: "success",
  error: true
}
```

এখন application confused হতে পারে।

Better design

```
{
  type: "success",
  data: [...]
}
```

অথবা:

```
{
  type: "error",
```

```
    message: "Failed"
  }
```

অথবা:

```
{
  type: "loading"
}
```

প্রতিটি state-এর নিজস্ব shape ।

State Machine-এর সাথে relation

এটা আগের design pattern topic-এর সাথেও related ।

ধরো:

```
Loading
  ↓
Success

Loading
  ↓
Error
```

Valid states:

```
Loading
Success
Error
```

Invalid combination:

```
Loading + Success + Error
```

Discriminated union এই valid states-কে explicit করে ।

Real API example

Backend থেকে:

```
{
  type: "success",
  data: {
    id: 10,
    name: "Rahim"
  }
}
```

Error:

```
{
  type: "error",
  code: "USER_NOT_FOUND",
  message: "User not found"
}
```

Frontend:

```
function handleUserResponse(response) {
  switch (response.type) {
    case "success":
      console.log(response.data.name);
      break;

    case "error":
      console.error(response.message);
      break;
  }
}
```

Discriminated Union + JSDoc

JavaScript-এ JSDoc দিয়ে এই design আরও clearly document করা যায়।

```
/**
 * @typedef {{type: "success", data: User}} SuccessResponse
```

```

* @typedef {{type: "error", message: string}} ErrorResponse
* @typedef {{type: "loading"}} LoadingResponse
*/

/**
* @typedef {SuccessResponse | ErrorResponse | LoadingResponse} Response
*/

```

তারপর:

```

/**
* @param {Response} response
*/
function handleResponse(response) {
  switch (response.type) {
    case "success":
      console.log(response.data);
      break;

    case "error":
      console.log(response.message);
      break;

    case "loading":
      console.log("Loading...");
      break;
  }
}

```

এখানে JSDoc documentation + discriminated union pattern একসাথে কাজ করছে।

Exhaustiveness Checking Concept

বড় application-এ একটা সমস্যা হয়:

```

switch (response.type) {
  case "success":
    // ...
    break;

  case "error":
    // ...

```

```
    break;  
}
```

পরে নতুন state যোগ করলে:

```
type = "empty"
```

কিন্তু handler update না করলে bug হতে পারে।

TypeScript এ এটা আরও strongly enforce করা যায়।

JavaScript-এও তুমি explicit fallback রাখতে পারো:

```
default:  
  throw new Error(`Unknown response type: ${response.type}`);
```

এটা production debugging-এ useful।

Discriminated Union কোথায় ব্যবহার করবে?

বিশেষ করে:

API responses

```
success  
error  
loading
```

Authentication

```
authenticated  
unauthenticated
```

Payment

```
pending  
success
```

```
failed
cancelled
```

File upload

```
idle
uploading
success
error
```

Game state

তোমার game-এর মতো application-এ:

```
waiting
night
day
voting
finished
```

এই pattern খুব useful ।

92. Runtime Type Validation — Zod, Yup Concepts

এটাই পুরো section-এর সবচেয়ে important concept ।

Static type information vs runtime data

ধরো:

```
/**
 * @typedef {Object} User
 * @property {string} name
 * @property {number} age
 */
```

এটা বলছে:

Developer expects User-এর shape এমন হবে ।

কিন্তু server থেকে যদি আসে:

```
{  
  "name": 123,  
  "age": "hello"  
}
```

JSDoc সেটা magically fix করবে না।

কারণ runtime data:

```
External World  
  ↓  
Unknown
```

External data কেন untrusted?

Application-এর বাইরে থেকে data আসতে পারে:

```
HTTP API  
Database  
User input  
URL  
Webhook  
File  
Environment variable  
Third-party API  
localStorage
```

এগুলোর shape assume করা dangerous।

Runtime validation

Runtime validation হলো:

```
Unknown Data  
  ↓  
Schema  
  ↓  
Valid?
```



Example without library

ধরো API থেকে user এসেছে:

```
const data = await response.json();
```

এখন directly:

```
console.log(data.name.toUpperCase());
```

করলে assumption হচ্ছে:

```
data.name → string
```

Better:

```
if (
  data !== null &&
  typeof data === "object" &&
  typeof data.name === "string" &&
  typeof data.age === "number"
) {
  console.log(data.name.toUpperCase());
}
```

কিন্তু complex schema হলে এই validation manually লেখা painful ।

এখানে Zod/Yup-এর মতো schema validation libraries useful ।

Zod Concept

Zod দিয়ে schema define করা যায় ।


```
import { z } from "zod";

const UserSchema = z.object({
  name: z.string(),
  age: z.number()
});
```

এখন:

```
const result = UserSchema.safeParse(data);
```

Result:

```
success = true
```

অথবা:

```
success = false
```

parse() vs safeParse()

parse()

```
const user = UserSchema.parse(data);
```

Valid হলে:

```
validated data
```

Invalid হলে:

```
throws error
```

safeParse()

```
const result = UserSchema.safeParse(data);
```

এটা exception না ছুড়ে result দেয়।

Conceptually:

```
if (result.success) {  
  console.log(result.data);  
} else {  
  console.error(result.error);  
}
```

এটা API validation-এর জন্য খুব convenient।

Why schema is powerful

Schema একইসাথে describe করে:

Expected shape
+
Runtime validation

Example:

```
const UserSchema = z.object({  
  id: z.number(),  
  name: z.string(),  
  email: z.string().email(),  
  active: z.boolean()  
});
```

এখানে:

id → number
name → string

```
email    → valid email
active   → boolean
```

Nested validation

Real application-এ nested data common।

```
const UserSchema = z.object({
  id: z.number(),
  profile: z.object({
    name: z.string(),
    age: z.number()
  })
});
```

এখন পুরো structure validate করা যায়।

Array validation

```
const UsersSchema = z.array(
  z.object({
    id: z.number(),
    name: z.string()
  })
);
```

Expected:

```
[
  { id: 1, name: "Rahim" },
  { id: 2, name: "Karim" }
]
```

Optional fields

Concept:

```
const UserSchema = z.object({
  name: z.string(),
  nickname: z.string().optional()
});
```

এখানে:

```
name → required
nickname → optional
```

Nullable vs Optional

এগুলো এক জিনিস নয়।

Optional:

```
property থাকতে নাও পারে
```

Nullable:

```
property আছে কিন্তু value null হতে পারে
```

Concept:

```
nickname: optional string
bio: string | null
```

এই distinction API design-এ খুব গুরুত্বপূর্ণ।

Enum-like validation

Discriminated union-এর সাথে খুব useful:

```
const RoleSchema = z.enum([
  "admin",
  "user",
```

```
"guest"  
  ]);
```

এখন:

```
admin ✓  
user ✓  
guest ✓  
owner ✗
```

Transform / Normalize concept

Runtime validation-এর পর data normalize করাও useful হতে পারে।

যেমন:

```
Input  
↓  
Validate  
↓  
Normalize  
↓  
Application
```

ধরো API date string পাঠালো:

```
"2026-09-07"
```

Application প্রয়োজন অনুযায়ী এটাকে Date/object বা normalized representation-এ convert করতে পারে।

তবে validation আর transformation-এর responsibility আলাদা করে রাখা ভালো।

Yup Concept

Yup-ও schema validation library।

Concept:

```
const schema = yup.object({
  name: yup.string().required(),
  age: yup.number().required()
});
```

তারপর schema দিয়ে input validate করা যায়।

Zod vs Yup — Conceptual Difference

দুটাই schema validation-এর জন্য useful।

বিষয়	Zod	Yup
Schema-based validation	✓	✓
Runtime validation	✓	✓
Object validation	✓	✓
Nested validation	✓	✓
Form validation	✓	✓
TypeScript integration	Strong	Strong
Transformation	✓	✓

Practical choice project ecosystem-এর ওপর depend করে।

React forms-এর ecosystem-এ Yup historically খুব common, আর modern TypeScript-heavy application-এ Zod অনেক popular।

Runtime Validation কোথায় করব?

সবচেয়ে important হলো **boundary**-তে validate করা।

API boundary

```
External API
  ↓
Validate
```

↓
Application

User input

Form
↓
Validate
↓
Backend

Backend request

Node.js:

HTTP Request
↓
Parse
↓
Validate
↓
Authorization
↓
Business Logic

Environment variables

এটাও খুব useful ।

process.env
↓
Unknown strings
↓
Validate
↓
Application config

কারণ:

```
process.env.PORT
```

সাধারণত `string` হিসেবে আসে।

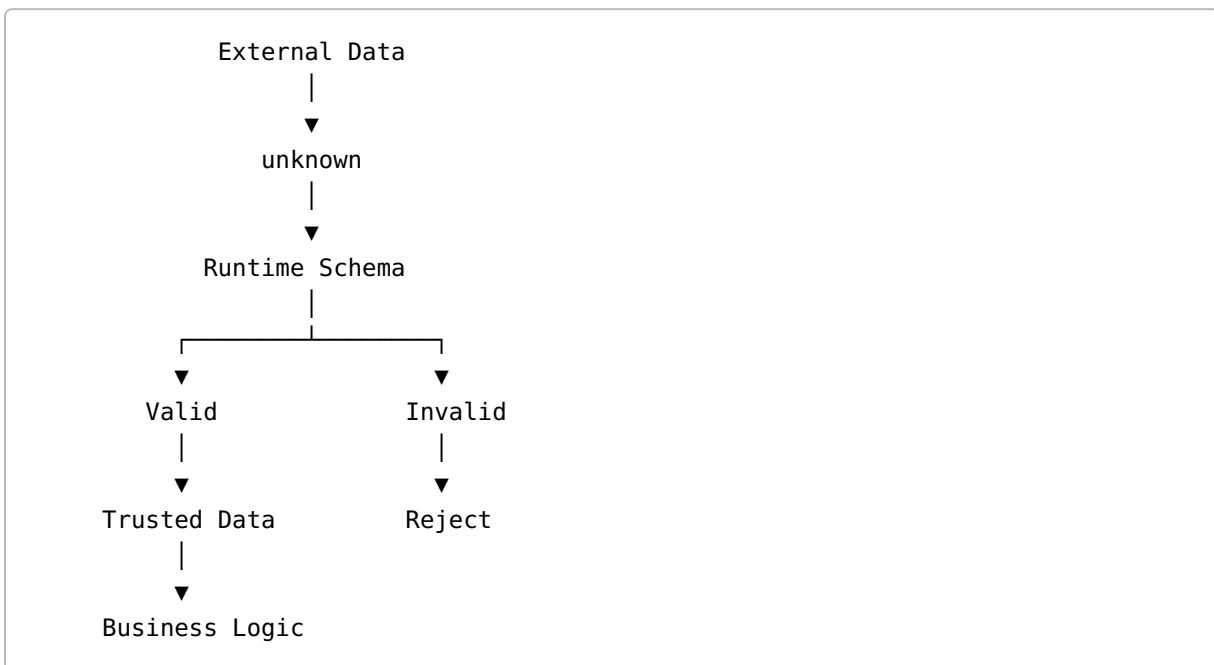
তুমি যদি ধরে নাও:

```
const port = process.env.PORT;
```

এটা `number`—ভুল assumption হতে পারে।

Runtime Validation + Type Safety

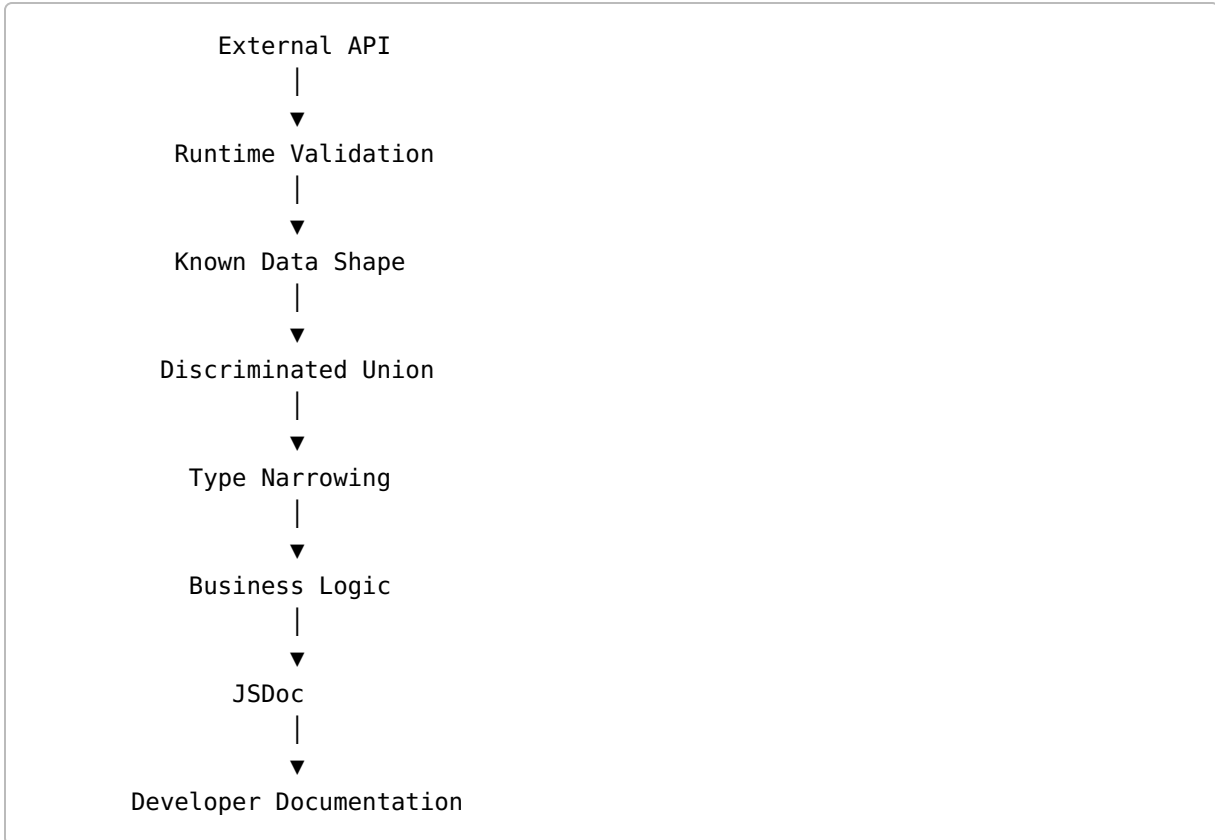
পুরো flow:



এটাই production-grade pattern।

JSDoc + Narrowing + Discriminated Union + Runtime Validation

চারটা topic একসাথে দেখো:



এখানে প্রত্যেকটির আলাদা responsibility আছে।

JSDoc বনাম Runtime Validation

এটা অবশ্যই মনে রাখবে:

	JSDoc	Runtime Validation
Runtime-এ execute হয়?	✗	✓
Developer documentation	✓	✓
Editor support	✓	কিছুটা
External data verify	✗	✓
API response validate	✗	✓
Wrong data reject	✗	✓

সহজভাবে:

JSDoc tells developers what should happen. Runtime validation checks what actually happened.

Type Narrowing বনাম Runtime Validation

দুটো related কিন্তু identical নয়।

ধরো:

```
if (typeof value === "string") {  
  // narrowed to string  
}
```

এটা simple runtime check।

কিন্তু:

```
API response  
↓  
50 fields  
↓  
nested objects  
↓  
arrays  
↓  
optional values  
↓  
validation library
```

এখানে schema validation অনেক বেশি practical।

Production API Example

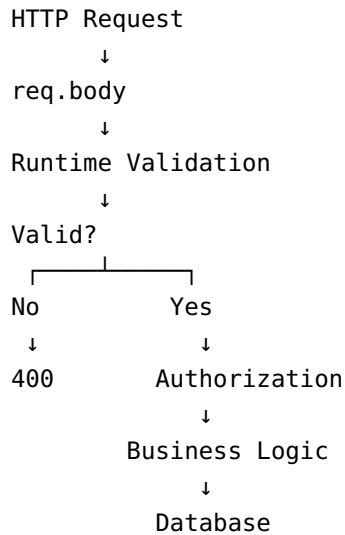
ধরো Node.js backend-এ:

```
POST /users
```

Client পাঠালো:

```
{
  "name": "Rahim",
  "age": 30,
  "email": "rahim@example.com"
}
```

Backend flow:



সবচেয়ে important:

Database -এর আগে validation

কিন্তু শুধু database-এর আগে নয়—**security-sensitive operation**-এর আগে **authorization**-ও করতে হবে।

Validation Error Handling

Bad:

```
try {
  const user = UserSchema.parse(req.body);
} catch (error) {
  res.status(500).json({
    message: error.message
  });
}
```

```
});  
}
```

Invalid user input সাধারণত server failure না।

Better conceptual distinction:

```
Invalid input  
  ↓  
4xx  
  
Server failure  
  ↓  
5xx
```

তবে exact HTTP status এবং error structure API design অনুযায়ী নির্ধারণ করবে।

Do Not Trust Client Validation

Frontend:

```
React form  
  ↓  
Zod/Yup validation
```

ভালো।

কিন্তু backend:

```
Client validation
```

এর ওপর trust করবে না।

কারণ attacker সরাসরি API call করতে পারে।

Correct:

Frontend validation
+
Backend validation

Frontend validation:

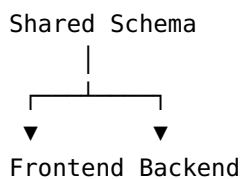
UX

Backend validation:

Security + correctness

Schema Reuse

একই schema frontend/backend share করার architecture কিছু project-এ useful হতে পারে।



তবে blindly shared codebase বানানোর দরকার নেই। Server-specific security/business rules server-side-এই থাকতে হবে।

Common Mistakes

✗ JSDoc লিখলেই runtime data safe ভাবা

```
/**  
 * @param {User} user  
 */  
function processUser(user) {}
```

এটা runtime validation নয়।

✗ শুধু frontend validation

Attacker frontend bypass করতে পারে।

✗ `typeof object` দিয়ে **complex validation** শেষ মনে করা

```
typeof data === "object"
```

এটা বলে না object-এর structure correct কিনা।

✗ `instanceof` দিয়ে **API JSON validate** করার চেষ্টা

API থেকে পাওয়া JSON সাধারণ object:

```
const data = await response.json();
```

এটা কোনো custom class-এর instance হওয়া জরুরি নয়।

তাই:

```
data instanceof User
```

সাধারণ JSON response-এর shape validation-এর equivalent নয়।

✗ **Optional** এবং **nullable confuse** করা

optional → property missing হতে পারে

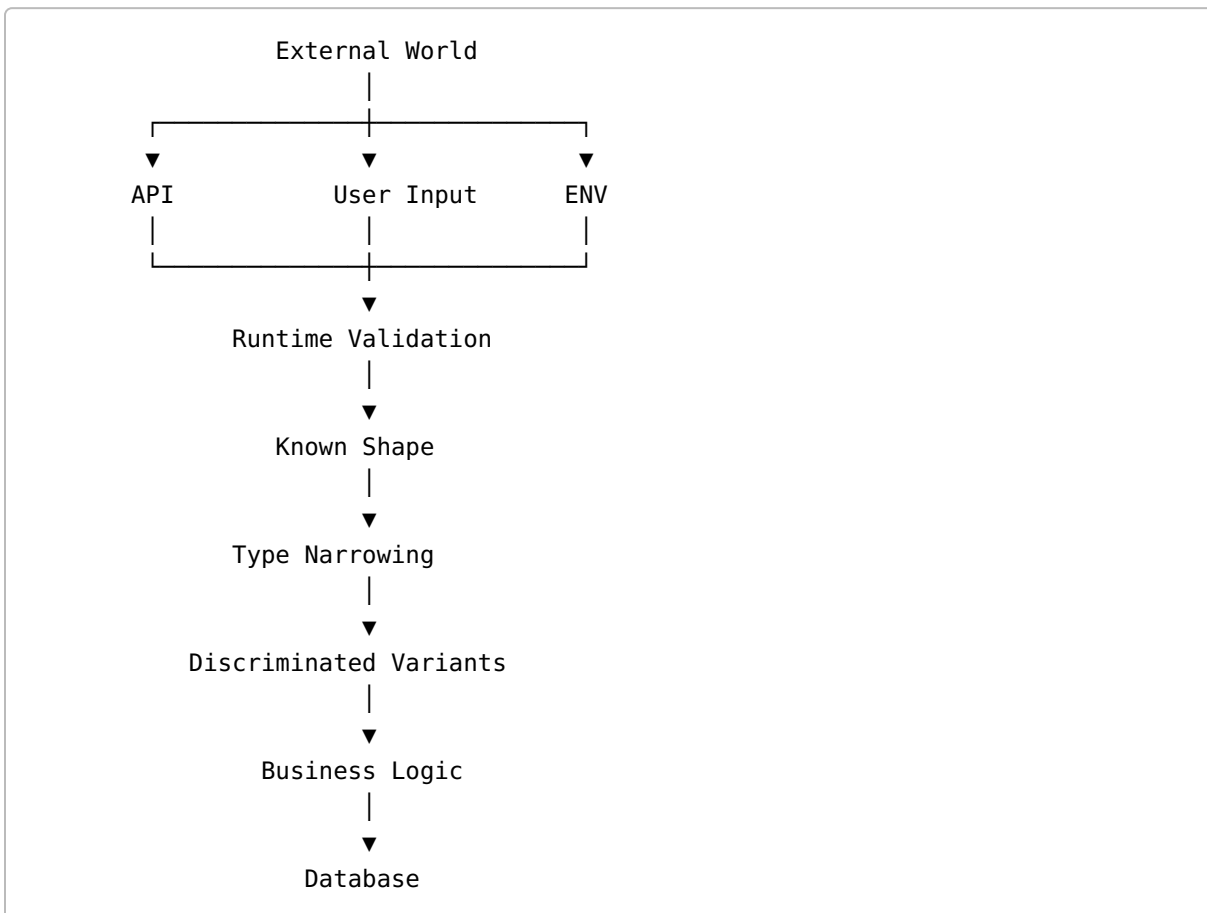
nullable → property exists, value null হতে পারে

✗ **Validation-এর পরেও any-like assumptions**

Validation boundary-এর পরে validated shape maintain করতে হবে।

Senior-Level Architecture

একটি বড় JavaScript/Node.js application-এ type safety architecture:



Documentation side:

```
JSDoc
↓
Functions
↓
Parameters
↓
Return values
↓
Object shapes
↓
Developer understanding
```

Practical Rules

Rule 1

External data:

```
Treat as untrusted
```

Rule 2

Simple type:

```
typeof
```

Rule 3

Class/object identity:

```
instanceof
```

Rule 4

Property-based variant:

```
in
```

Rule 5

Array:

```
Array.isArray()
```

Rule 6

Complex external structure:

```
Runtime schema validation
```


Rule 7

Multiple object variants:

Discriminated Union

Rule 8

Developer documentation:

JSDoc

Rule 9

Frontend validation is not backend security.

Client validation $\neq$ Server validation

Rule 10

Type safety এবং security একই জিনিস নয়।

Type safety
↓
Correctness

Security
↓
Protection against malicious behavior

দুটো overlap করে, কিন্তু এক নয়।

Quick Revision

JSDoc
↓
Describe JavaScript types

typeof
↓

Primitive type narrowing

`instanceof`

↓

Class / constructor identity

`in`

↓

Property-based narrowing

Discriminated Union

↓

Object variant identified by type/kind/status

Runtime Validation

↓

Verify actual external data

Zod / Yup

↓

Schema-based runtime validation

সবচেয়ে important distinction:

JSDoc

"What should this data look like?"

Runtime validation

"Does this data actually look like that?"

Interview Questions

Beginner

1. JSDoc কী?
2. `@param` এবং `@returns` কী?
3. `typeof` কী return করে?
4. `typeof null` কেন `"object"`?
5. `instanceof` কী?
6. `in` operator কী করে?

Intermediate

1. Type narrowing কী?
2. `Array.isArray()` কেন দরকার?
3. Discriminated union কী?
4. `type` property discriminator হিসেবে কীভাবে কাজ করে?
5. JSDoc কি runtime validation করে?
6. Zod/Yup কেন ব্যবহার করা হয়?
7. `optional` এবং `nullable` -এর পার্থক্য কী?
8. Frontend validation থাকা সত্ত্বেও backend validation কেন দরকার?

Senior

1. External API response-কে trusted data হিসেবে কখন treat করবে?
2. Runtime validation application-এর কোন boundary-তে করা উচিত?
3. Discriminated union কীভাবে impossible states কমাতে সাহায্য করে?
4. JSDoc + runtime schema validation কীভাবে একসাথে ব্যবহার করবে?
5. `instanceof` কেন সাধারণ JSON API response validation-এর জন্য যথেষ্ট নয়?
6. Node.js API-তে request validation, authentication এবং authorization-এর order কীভাবে design করবে?
7. Shared schema frontend/backend-এ ব্যবহার করার সুবিধা ও trade-off কী?
8. Type safety এবং security-এর মধ্যে পার্থক্য কী?

Practice Tasks

Task 1 — JSDoc

একটি `UserService` বানাও এবং JSDoc দিয়ে document করো:

```
User
├── id
├── name
├── email
└── active
```

তারপর:

```
createUser()
getUser()
updateUser()
```

প্রতিটির parameter এবং return type document করো।

Task 2 — Type Narrowing

একটি function বানাও:

```
processValue(value)
```

যেটা handle করবে:

```
string  
number  
boolean  
array  
null  
object
```

`typeof`, `Array.isArray()` এবং null checking ব্যবহার করো।

Task 3 — Discriminated Union

একটি payment response design করো:

```
pending  
success  
failed  
cancelled
```

প্রতিটি state-এর জন্য আলাদা object shape বানাও।

তারপর:

```
handlePayment(response)
```

function দিয়ে `switch` ব্যবহার করে প্রতিটি state handle করো।

Task 4 — Runtime Validation

একটি user API response ধরো:

```
{  
  "id": 10,  
  "name": "Rahim",  
  "email": "rahim@example.com",  
  "age": 30  
}
```

প্রথমে manual validation করো।

তারপর একই validation schema library দিয়ে implement করো।

Task 5 — Production API

একটি endpoint design করো:

POST /users

Flow:

```
Request  
↓  
Validate body  
↓  
Authenticate  
↓  
Authorize  
↓  
Business Logic  
↓  
Database  
↓  
Response
```

প্রতিটি step কেন দরকার সেটা নিজে explain করো।

Mastery Checklist

এই section সত্যি master করতে পারলে তুমি explain করতে পারবে:

- [] JSDoc কী এবং কেন ব্যবহার করা হয়

- [] @param, @returns, @typedef
- [] JSDoc-এর limitation
- [] typeof narrowing
- [] typeof null issue
- [] instanceof
- [] in
- [] Array.isArray()
- [] Custom type guard concept
- [] Discriminated union
- [] Discriminator কী
- [] State modeling-এর সাথে discriminated union-এর relation
- [] Exhaustiveness-এর concept
- [] Runtime validation কেন দরকার
- [] Zod/Yup-এর purpose
- [] parse() বনাম safeParse() concept
- [] Optional বনাম nullable
- [] Nested schema validation
- [] API boundary validation
- [] Environment variable validation
- [] Frontend বনাম backend validation
- [] Type safety বনাম security
- [] JSDoc + narrowing + schema validation একসাথে কীভাবে কাজ করে

Final Mental Model

শেষে পুরো section-টাকে এক লাইনে মনে রাখো:

```

DOCUMENT
  ↓
JSDoc

UNDERSTAND
  ↓
Type Narrowing

MODEL
  ↓
Discriminated Unions

VERIFY
  ↓
Runtime Validation
  
```

আর production application-এ:

```
External Data
↓
"Don't trust it"
↓
Validate
↓
Narrow / classify
↓
Known data shape
↓
Business logic
```

সবচেয়ে গুরুত্বপূর্ণ কথা:

JavaScript-এর **dynamic nature eliminate** করার চেষ্টা না করে, যেখানে **uncertainty** আছে সেখানে **boundary** তৈরি করাই **mature approach**।

```
Unknown
↓
Validation Boundary
↓
Known
↓
Business Logic
```

এই mental modelটা Node.js API, React application, forms, database layer, third-party APIs—সব জায়গাতেই কাজে লাগবে।